The enqueue/1 function isn't standardized by Oban, but it provides a convenient way to enqueue jobs based on known arguments. In our case, an SmsMessage struct is turned into a set of parameters that's passed into the job.

Oban uses Ecto changesets and repo functions to insert jobs. The new/1 call turns arguments into an Oban.Job changeset, and then the call to Oban.insert/1 places that job in the database.

Arguments in Oban have similar limitations to Sidekiq. It's a best practice to only pass identifiers into the job arguments—as opposed to complex structs or maps—and load the data based on those identifiers in the job itself. Here's what that looks like when we write the perform/1 function:

phone_app/lib/phone_app/conversations/worker/status_worker.ex

```
alias PhoneApp.Conversations.Query.SmsMessageStore

def perform(%Oban.Job{args: %{"id" => message_id}}) do
  message = SmsMessageStore.get_sms_message!(message_id)
  %{body: resp} = PhoneApp.Twilio.get_sms_message!(message)

  case resp["status"] do
    "queued" ->
      {:error, "Message not ready"}

    status ->
      PhoneApp.Conversations.update_sms_message(
        message.message_sid,
        %{status: status}
      )
  end
end
```

This job follows a simple formula: load the data from the arguments, query the Twilio API for the current status, and then persist that status into the database. The perform/1 function can return a variety of possible results, which are all documented in the Oban documentation.[6] The update_sms_message/2 function returns either an ok or error tuple, so it works perfectly as a return value.

If we detect that the message still isn't ready, then we cause the job to fail and retry by returning {:error, reason}. Oban provides a default retry logic that's similar to Sidekiq—twenty retries over an exponential backoff period. (Like most things, you have complete control over the retry logic.)

If you tried to start this now, you'd get an error about SmsMessageStore.get_sms_message!/1 not existing. Let's add that quickly:

6. https://hexdocs.pm/oban/Oban.Worker.html#t:result/0